COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S4
DATE PERFORMED:	July 29, 2026	**DATE SUBMITTED:**	August 5, 2026
NAME:	Uchi, Mao Sophie S.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 4.1 **Stack**

1. PROCEDURE OUTPUT

ILO A: Create a stack using the C++ STL

Code

ilo_a.cpp | ilo_b1.cpp | ilo_b2.cpp

ilo_a.cpp > main()

```cpp
#include <iostream>
#include <stack>

int main(){

    // push

    // pop

    // isEmpty

    // isFull

    // Top

    // create an object:
    std::stack<char> stack1;
    std::stack<char> stack2;

    std::cout << "Is the stack empty? " << stack1.empty() << std::endl;
    stack1.push('j');
    std::cout << "Is the stack empty? " << stack1.empty() << std::endl;
    std::cout << "Current top of the stack: " << stack1.top() <<std::endl;
    std::cout <<stack1.size();
    stack1.push('i');
    std::cout << "Current top of the stack: " << stack1.top() <<std::endl;
    stack1.push('m');
    std::cout << "Current top of the stack: " << stack1.top() <<std::endl;
    std::cout << "Is the stack empty? " << stack1.empty() << std::endl;
    std::cout << "What is the size of the stack? " << stack1.size() << std::endl;
```

```
31
32
33      // swap testing
34      stack2.push('c');
35      stack2.push('o');
36      stack2.push('w');
37      std::cout << "\nBefore Swap: " <<std::endl;
38      std::cout << "Current top of the stack1: " << stack1.top() <<std::endl;
39      std::cout << "Current top of the stack2: " << stack2.top() <<std::endl;
40      stack1.swap(stack2);
41      std::cout << "\nAfter Swap: " <<std::endl;
42      std::cout << "Current top of the stack1: " << stack1.top() <<std::endl;
43      std::cout << "Current top of the stack2: " << stack2.top() <<std::endl;
44
45      // emplace testing
46      std::cout << "\nBefore Emplace: " <<std::endl;
47      std::cout << "Current top of the stack: " << stack1.top() <<std::endl;
48      stack1.emplace('k');
49      std::cout << "\nAfter Emplace: " <<std::endl;
50      std::cout << "Current top of the stack: " << stack1.top() <<std::endl;
51
52      return 0;
53  }
54
```

Output

```
Is the stack empty? 1
Is the stack empty? 0
Current top of the stack: j
1Current top of the stack: i
Current top of the stack: m
Is the stack empty? 0
What is the size of the stack? 3

Before Swap:
Current top of the stack1: m
Current top of the stack2: w
```

```
After Swap:
Current top of the stack1: w
Current top of the stack2: m

Before Emplace:
Current top of the stack: w

After Emplace:
Current top of the stack: k
PS D:\Downloads (HDD)\CPE010\HOA4>
```

Analysis

From our preset code, we have created 5 basic stack operation which are Push, Pop, isEmpty, isFull and Top. Operation Push, will add your input element into your stack up until your stack is full and might overflow. Operation Pop will remove your input element out of your stack until your stack is empty and might underflow. Operation isEmpty depends whether your stack is empty and will return as true but if the stack is NOT empty,

it will return as false. Operation isFull depends whether your stack is full and will return as true but if the stack is NOT full, it will return as false. Operation Top, depends on which is the latest pushed element, will print out that element. Now for our 2 additional basic stack operation, we have Stack Swap is one of the basic stack operators, used for switching 2 different stacks, taking place on each other's elements with their own elements. Stack Emplace is also one of the basic stack operators, used for replacing the top stack with a new top stack, completely overwriting the previous top stack, similar to Insert Head when replacing head of the nodes. DisplayAll function is used to show the data of the stacks in an order when you pushed or popped an element, similar to Top stack, but shows the whole stack of elements that had been pushed.

ILO B: Develop C++ code that uses both arrays and linked lists to create a stack

B.1. Stacks using Arrays

Code

ilo_a.cpp | ilo_b1.cpp ● | ilo_b2.cpp

ilo_b1.cpp > displayAll()

```cpp
#include <iostream>

//global variables:
#define maxSize 10
//const size_t maxSize = 10
//const int maxSize = 10
int stackArr[maxSize];
int top = -1, newData;

//function prototypes:
// Top, Push, Pop, isEmpty, isFull

// observer operator
bool isEmpty();
bool isFull();
void Top(); // the T is capital
void displayAll(); // goal is to display the stack following the LIFO principle

// transformer operator
void push();
void pop();


```

```cpp
int main(){
    int choice;
    while (true) {
        std::cout << "==========================\n";
        std::cout << "Stack Operatoins: " <<std::endl;
        std::cout << "[1] Push [2] Pop [3] isEmpty [4] isFull [5] Top [6] Display All" << std::endl;
        std::cin >> choice;
        switch(choice) {

            case 1: push();
            break;
            case 2: pop();
            break;
            case 3: std::cout << "is the stack empty? " << isEmpty() << std::endl;
            break;
            case 4: std::cout << "is the stack full? " << isFull() << std::endl;
            break;
            case 5: Top();
            break;
            case 6: displayAll();
            break;
            default: std::cout << "Invalid Choice! " <<std::endl;
        }
    }
    return 0;
}
```

```cpp

//function definition
bool isEmpty(){
    return (top == -1);
}

bool isFull(){
    return (top == maxSize-1);
}

void Top(){
    //error catching:
    if (isEmpty()) {
        std::cout << "Stack Underflow! " <<std::endl;
        return;
    }

    //prints the top element
    std::cout << "The top element is: " << stackArr[top] << std::endl;
}

```

```
71  void push(){
72      //error catching
73      if (isFull()) {
74          std::cout << "Stack Overflow! " <<std::endl;
75          return;
76      }
77
78      //ask the user what they want to insert:
79      std::cout << "What character do you want to push? ";
80      std::cin >> newData;
81
82
83      //storing to the stack:
84      stackArr[++top] = newData;
85
86  }
87
88  void pop(){
89      // error catching:
90      if (isEmpty()) {
91          std::cout << "Stack Underflow! " <<std::endl;
92      }
93
94      // display the item to be popped:
95      std::cout << "Popping: " << stackArr[top] <<std::endl;
96
97      // we remove the element from the stack
98      top--;
99
100 }
```

```
101
102   // Displaying Stack's Elements
103   void displayAll() {
104       // error catching:
105       if (top == -1) {
106           std::cout << "Stack Underflow! " <<std::endl;
107           return;
108       }
109
110       std::cout << "\n";
111       for (int i = top; i >= 0; --i) {
112           std::cout << stackArr[i] << std::endl;
113       }
114       return;
115   }
```

Output

```
============================
Stack Operatoins:
[1] Push [2] Pop [3] isEmpty [4] isFull [5] Top [6] Display All
1
What character do you want to push? 2
============================
Stack Operatoins:
[1] Push [2] Pop [3] isEmpty [4] isFull [5] Top [6] Display All
1
What character do you want to push? 4
============================
Stack Operatoins:
[1] Push [2] Pop [3] isEmpty [4] isFull [5] Top [6] Display All
1
What character do you want to push? 6
============================
Stack Operatoins:
[1] Push [2] Pop [3] isEmpty [4] isFull [5] Top [6] Display All
6

6
4
2
============================
```

B.2. Stacks using Linked Lists

Code

ilo_b2.cpp ● ilo_c2.cpp ● stackList.h ●

ilo_b2.cpp > ...

```cpp
#include <iostream>
#include "stackList.h"

int main() {

    std::cout << " >>> Pushing 'C' 'P' 'E' onto the stack <<< \n";
    //push
    push('C');
    push('P');
    push('E');

    //display the current top
    Top<char>(); // Top should be 'e'
    displayAll<char>();
    std::cout << "\n >>> Popping all the elements <<< \n";

    while(!isEmpty<char>()){
        std::cout <<" Popped: " << pop<char>() << std::endl;
    }

    std::cout << "\n>>> Testing the Underflow <<< \n";
    pop<char>();

    return 0;
}
```

stackList.h ● ilo_b2.cpp ● ilo_c2.cpp ●

stackList.h > ...

```cpp
#ifndef STACK_LIST_H
#define STACK_LIST_H

// class nothe that contains data and the next
template <typename T>
class Node {
    public:
    T data;          // VALUE THAT WE NEED STORE
    Node* next;      // POINTER TO THE NEXT NODE
};

//global pointer for the top of the stack
template <typename T>
Node<T> *head = nullptr;

// operations for the stack
// push
template <typename T>
void push(T newData){

    Node<T>* newNode = new Node<T>;
    newNode->data = newData;

    newNode->next = head<T>;
    head<T> = newNode;
};

//isEmpty
template<typename T>
bool isEmpty(){
    return head<T> == nullptr;
}
```

```cpp

//pop
template <typename T>
T pop(){
    //check if the stack is empty
    if(head<T> == nullptr){
        std::cout << "Stack overflow" << std::endl;
    }

    //save the current top node
    Node<T>* temp = head<T>;
    //store its data
    T tempVal = temp->data;

    // move the head to the next node
    head<T> = head<T>->next;
    // free the removed node
    delete temp;
    //return the removed node
    return tempVal;
}

//Top
template <typename T>
void Top(){

    if(isEmpty<T>()){
        std::cout << "[ Stack underflow ]" <<std::endl;
        return;
    }

    std::cout <<"Top of the Stack: " << head<T>->data << std::endl;
}

```

```cpp
//displayAll
template <typename T>
void displayAll(){
    if(head<T> == nullptr) {
        std::cout << "Stack underflow\n";
        return;
    }
    std::cout << "Displaying the stack: \n";
    Node<T>* temp = head<T>;
    while(temp != nullptr){
        std::cout << temp->data << "\n";
        temp = temp->next;
    }
    std::cout <<"\n";
}

#endif
```

Output

```
>>> Popping all the elements <<<
Popped: E
Popped: P
Popped: C

>>> Testing the Underflow <<<
Stack overflow
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve problems using an implementation of stack:

Stack using Arrays
Code

ilo_c.cpp > balSymbols(std::string)

```cpp
#include <iostream>
#include <string>

class Array {
public:
    char maxArray[100];
    int top;

    Array() {
        top = -1;
    }
    bool isEmpty() {
        return top == -1;
    }
    bool isFull() {
        return top == 100 - 1;
    }
    void push(char character) {
        if (!isFull()) {
            maxArray[++top] = character;
        }
    }
    char pop() {
        if (!isEmpty()) {
            return maxArray[top--];
        }
        return '\0';
    }
};

```

```cpp
bool symbols(char open, char close) {
    return (open == '(' && close == ')') || (open == '{' && close == '}') || (open == '[' && close == ']');
}

bool pushSymbols(std::string expression) {
    Array stack;
    for (char character : expression) {
        if (character == '(' || character == '{' || character == '[') {
            stack.push(character);
        }
        else if (character == ')' || character == '}' || character == ']') {
            if (stack.isEmpty())
                return false;
            char top = stack.pop();
            if (!symbols(top, character))
                return false;
        }
    }
    return stack.isEmpty();
}
```

```
51
52  int main() {
53      int choice;
54      while (true) {
55      std::cout << "Stack using Array: " << std::endl;
56      std::cout << "[1] Checking Expression [2] Exit : ";
57      std::cin >> choice;
58      switch(choice){
59          case 1: {
60          std::string expression;
61          std::cout << "\nInput an expression: ";
62          std::cin >> expression;
63
64          if (pushSymbols(expression))
65              std::cout << ">>> Valid Expression!\n" << std::endl;
66          else
67              std::cout << ">>> Not Valid Expression!\n" << std::endl;
68          break;
69          }
70
71          case 2:
72          return 0;
73
74          default:
75          std::cout << "Try Again!\n" << std::endl;
76          }
77      }
78      return 0;
79  }
```

Stack using Linked Lists
Code

```
15  class linkListStack {
16  public:
17      Node* top;
18      linkListStack() {
19          top = NULL; // Set the stack empty
20      }
21      void push(char character) {
22          Node* newNode = new Node(character);
23          newNode->next = top;
24          top = newNode;
25      }
26      char pop() {
27          if (isEmpty()) return '\0'; // Character will be null
28          // Replicating Pop
29          Node* temp = top;
30          char value = temp->data;
31          top = top->next;
32
33          delete temp;
34          return value;
35      }
36      bool isEmpty() {
37          return top == NULL;
38      }
39  };
```

Expression	Valid (Y/N)	Output	Analysis
(A+B) + (C-D)	Y	Stack using Array: [1] Checking Expression [2] Exit : 1 Input an expression: (A+B)+(C-D) >>> Valid Expression!	Both A+B and C-D are enclosed with an open parenthesis and closed parentesis, making this a valid expression.

((A+B) + (C-D)	N	Stack using Array: [1] Checking Expression [2] Exit : 1 Input an expression: ((A+B)+(C-D) >>> Not Valid Expression!	C-D are enclosed with an open parenthesis and closed parenthesis, but A+B having an extra open parenthesis, making this not a valid expression.
((A+B) + [C-D])	Y	Stack using Array: [1] Checking Expression [2] Exit : 1 Input an expression: ((A+B)+[C-D]) >>> Valid Expression!	Both A+B and C-D are enclosed with 3 open parenthesis and 3 closed parenthesis, making this a valid expression.
((A+B] + [C-D]}	N	Stack using Array: [1] Checking Expression [2] Exit : 1 Input an expression: ((A+B]+[C-D]} >>> Not Valid Expression!	Both A+B and C-D may have an equal amount of parenthesis, but each have incorrect pairs of their parenthesis, making this not a valid expression.

Conclusion (edit)
My conclusion for this activity is I've learn how to use the 7 basic stack operations in ILO A, Operation Push adds an input element to a stack until it reaches capacity, possibly causing overflow. Operation Pop removes elements until the stack is empty, risking underflow. The isEmpty operation checks if the stack is empty, returning true if it is, and false otherwise. The isFull operation checks for fullness, returning true if full, and false otherwise. The Top operation displays the most recently added element. Additionally, Stack Swap allows for switching elements between two stacks, while Stack Emplace replaces the top element of a stack with a new one. The DisplayAll function shows all currently stored elements in the stack, reflecting the order in which they were added or removed. Hopefully these implementations I've done will help me in my future activities and make my progression in coding muc

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Quillbot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Summarization**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

COMPUTER ENGINEERING

QUEZON CITY

Extent of Use: **I'd like to re-explain my original thoughts and understanding in a shorter way**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.